Statistical and Machine Learning

Exemplar and Kernel Methods

Instructions

All questions should first be discussed in pairs or small groups. Afterwards, we shall discuss them together.

Exercise 1 — k-Nearest Neighbours: Classification and Regression

(a) A k-NN classifier estimates the class probabilities for a new point $\mathbf{x}$ using its k nearest neighbours $N_{\mathbf{x}}^{k}$:

$$\hat{P}(Y = l \mid \mathbf{x}) = \frac{1}{k} \sum_{i \in N_{\mathbf{x}}^{k}} \mathbf{1}_{\{y_i = l\}}.$$

Suppose $k = 5$ and the five nearest neighbours of a query point have labels $(+, +, -, +, -)$. Compute $\hat{P}(Y = + \mid \mathbf{x})$ and state the predicted class.

(b) For k-NN regression the prediction is

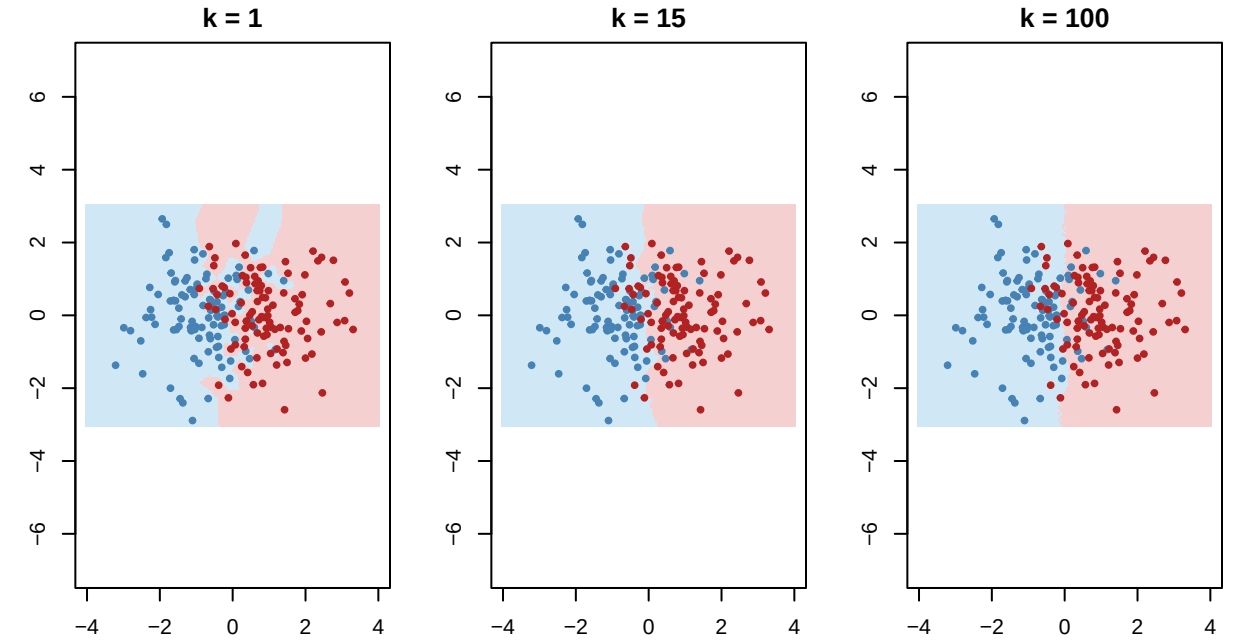
$$\hat{f}(\mathbf{x}) = \frac{1}{k} \sum_{i \in N_{\mathbf{x}}^{k}} y_i.$$

Using the same five neighbours as in (a), now with continuous responses $y = (3.2,\ 4.1,\ 1.0,\ 3.8,\ 2.5)$, compute $\hat{f}(\mathbf{x})$.

(c) The code below generates a binary classification dataset and fits k-NN for three values of k. Run the code, examine the decision boundaries, and answer: which value of k leads to under-fitting, which to over-fitting, and which provides the best trade-off? Explain how k controls the bias–variance trade-off in k-NN.

```r
library(class)

set.seed(1)
n     <- 200
x1    <- c(rnorm(n/2, -1), rnorm(n/2,  1))
x2    <- c(rnorm(n/2,  0), rnorm(n/2,  0))
y     <- factor(rep(c("A", "B"), each = n/2))
train <- data.frame(x1 = x1, x2 = x2, y = y)

# Prediction grid
grid  <- expand.grid(x1 = seq(-4, 4, length.out = 150),
                     x2 = seq(-3, 3, length.out = 150))

par(mfrow = c(1, 3), mar = c(3, 3, 2, 1))
for (k_val in c(1, 15, 100)) {
  pred <- knn(train[, 1:2], grid, train$y, k = k_val)
  plot(grid$x1, grid$x2, col = ifelse(pred == "A", "#d0e8f5", "#f5d0d0"),
       pch = 15, cex = 0.4,
```

```
       main = paste0("k = ", k_val),
       xlab = "x1", ylab = "x2", asp = 1)
  points(x1, x2, col = ifelse(y == "A", "steelblue", "firebrick"),
         pch = 19, cex = 0.6)
}
```




```
par(mfrow = c(1, 1))
```

(d) State one advantage and one disadvantage of k-NN compared to a parametric classifier such as logistic regression.

Exercise 2 — Kernel Density Estimation and Bandwidth Selection

(a) A univariate kernel density estimator (KDE) is defined as

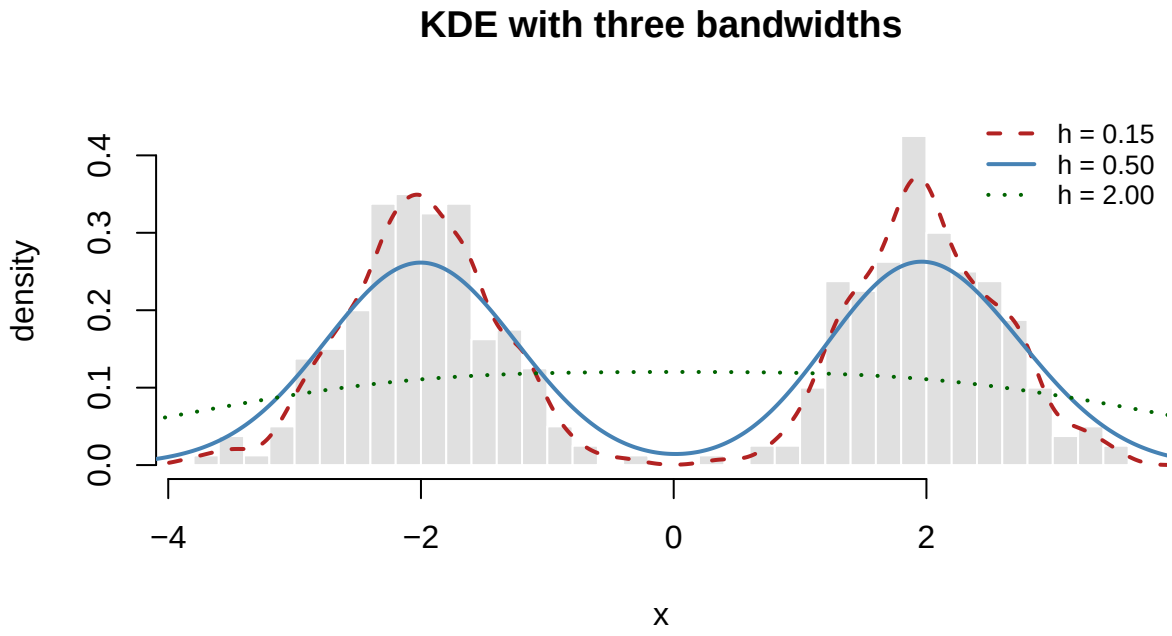
$$\hat{f}(x) = \frac{1}{nh} \sum_{i=1}^{n} K\left(\frac{x - X_i}{h} \right),$$

where K is a kernel function and $h > 0$ is the bandwidth. State the three properties a function K must satisfy to be a valid kernel (density) function, and verify that the Gaussian kernel $K(u) = \frac{1}{\sqrt{2\pi}} e^{-u^2/2}$ satisfies them.

(b) The code below generates data from a mixture of two Gaussians and fits KDEs with three different bandwidths. Run the code, then describe the effect of each bandwidth on the estimated density.

```
set.seed(42)
x <- c(rnorm(200, mean = -2, sd = 0.6),
       rnorm(200, mean =  2, sd = 0.6))

d_small  <- density(x, bw = 0.15)
d_medium <- density(x, bw = 0.50)
d_large  <- density(x, bw = 2.00)
```

```r
hist(x, breaks = 30, probability = TRUE,
     col = "grey88", border = "white",
     main = "KDE with three bandwidths",
     xlab = "x", ylab = "density",
     ylim = c(0, 0.45))
lines(d_small,  col = "firebrick",   lwd = 2, lty = 2)
lines(d_medium, col = "steelblue",   lwd = 2, lty = 1)
lines(d_large,  col = "darkgreen",   lwd = 2, lty = 3)
legend("topright",
       legend = c("h = 0.15", "h = 0.50", "h = 2.00"),
       col    = c("firebrick", "steelblue", "darkgreen"),
       lwd = 2, lty = c(2, 1, 3), bty = "n", cex = 0.8)
```



(c) Explain the bias–variance trade-off for the bandwidth h. What happens when h is too small or too large?

(d) Two principled methods for choosing h are **leave-one-out (LOO) cross-validation** and **data splitting**. Briefly explain the idea behind each and state what criterion is minimised in both cases.

Exercise 3 — Mercer Kernels and the Kernel Trick

(a) State the two conditions a function $K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ must satisfy to be a valid (Mercer) kernel, and explain what the **Gram matrix** is.

(b) Consider the degree-2 polynomial kernel $K(\mathbf{x}, \mathbf{z}) = \langle \mathbf{x}, \mathbf{z} \rangle^2$ for $\mathbf{x} = (x_1, x_2)^\top$ and $\mathbf{z} = (z_1, z_2)^\top$.

- Expand $K(\mathbf{x}, \mathbf{z})$ algebraically.
- Identify the explicit feature map $\phi(\mathbf{x})$ such that $K(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x})^\top \phi(\mathbf{z})$.
- What is the dimension of the feature space?

(c) The RBF (Gaussian) kernel is

$$K(\mathbf{x}, \mathbf{z}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{z}\|^2}{2\ell^2} \right).$$

Describe the effect of the length-scale ℓ on which pairs of points the kernel considers similar. What is unusual about the feature space that this kernel corresponds to?

(d) Explain the **kernel trick** in one or two sentences: what computational advantage does it provide, and in which part of an algorithm does it apply?

Exercise 4 — Gaussian Process Regression

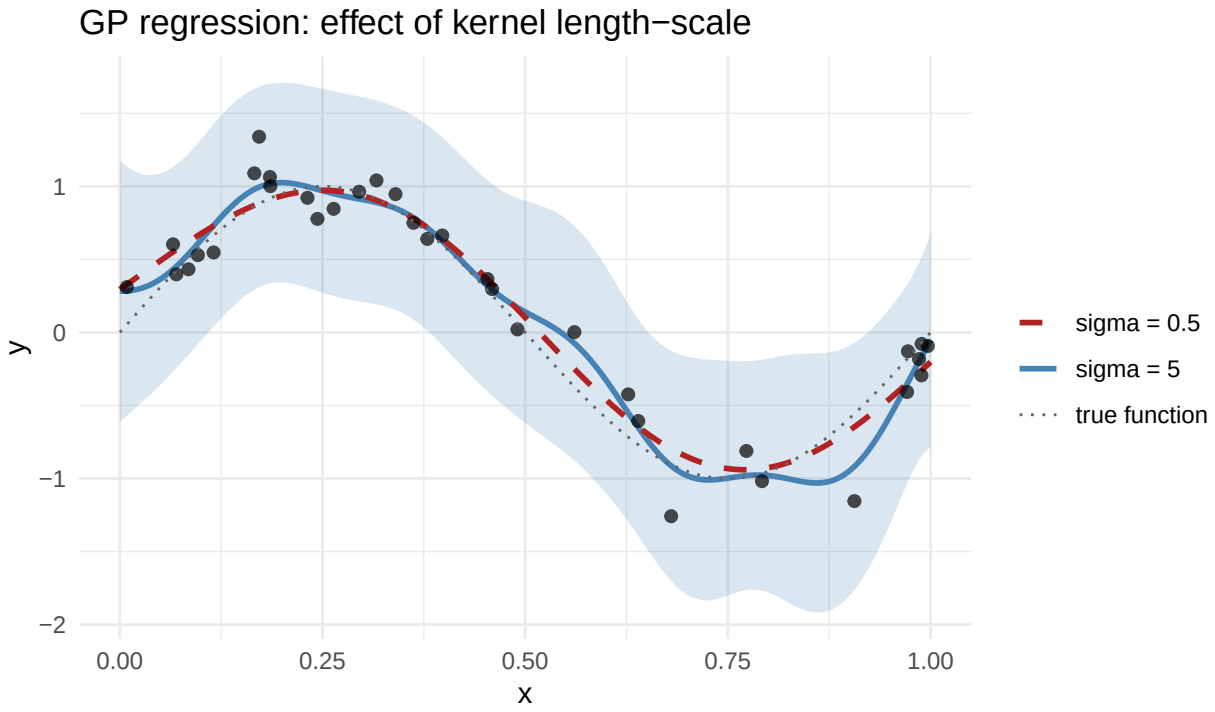
(a) A Gaussian process is written $f \sim \mathcal{GP}(m, \mathcal{K})$. Explain the role of the mean function $m(\mathbf{x})$ and the covariance (kernel) function $\mathcal{K}(\mathbf{x}, \mathbf{x}')$. What distribution does $\big(f(\mathbf{x}_1), \ldots, f(\mathbf{x}_n)\big)$ follow for any finite set of inputs?

(b) In **noisy** GP regression, observations are $y_i = f(\mathbf{x}_i) + \epsilon_i$ with $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$. Write down the covariance matrix of the observed vector $\mathbf{y} = (y_1, \ldots, y_n)^\top$ and explain why a noise-free GP would not be appropriate for most real datasets.

(c) The code below fits a GP regression model to noisy data using `gausspr()` from `kernlab`. Run the code and then answer the questions that follow.

```
library(kernlab)

set.seed(7)
n       <- 35
x_train <- sort(runif(n, 0, 1))
y_train <- sin(2 * pi * x_train) + rnorm(n, sd = 0.25)
x_grid  <- seq(0, 1, length.out = 300)

# Fit GP with RBF kernel
gp1 <- gausspr(x_train, y_train,
               kernel = "rbfdot",
               kpar   = list(sigma = 5),
               var    = 0.09,
               variance.model = TRUE)

gp2 <- gausspr(x_train, y_train,
               kernel = "rbfdot",
               kpar   = list(sigma = 0.5),
               var    = 0.09,
               variance.model = TRUE)

mu1 <- as.vector(predict(gp1, x_grid))
sd1 <- as.vector(predict(gp1, x_grid, type = "sdeviation"))
mu2 <- as.vector(predict(gp2, x_grid))

df_tr  <- data.frame(x = x_train, y = y_train)
df_grd <- data.frame(x  = x_grid,
                     mu1 = mu1,
                     lo1 = mu1 - 1.96 * sd1,
                     hi1 = mu1 + 1.96 * sd1,
                     mu2 = mu2)

ggplot(df_grd, aes(x = x)) +
  geom_ribbon(aes(ymin = lo1, ymax = hi1),
              fill = "steelblue", alpha = 0.2) +
  geom_line(aes(y = mu1, colour = "sigma = 5"),   linewidth = 1) +
  geom_line(aes(y = mu2, colour = "sigma = 0.5"), linewidth = 1,
            linetype = "dashed") +
```

```r
  geom_point(data = df_tr, aes(x, y),
             colour = "black", size = 1.8, alpha = 0.7) +
  geom_line(aes(y = sin(2 * pi * x), colour = "true function"),
            linetype = "dotted") +
  scale_colour_manual(values = c("sigma = 5"     = "steelblue",
                                 "sigma = 0.5"   = "firebrick",
                                 "true function" = "grey40")) +
  labs(title = "GP regression: effect of kernel length-scale",
       x = "x", y = "y", colour = "") +
  theme_minimal()
```



- Which model (`sigma = 5` or `sigma = 0.5`) fits more closely to the data and which is smoother? Explain why.
- Where is the posterior uncertainty (shaded band) largest, and what does this reflect about the GP?
- The true function is $\sin(2\pi x)$ with noise SD $= 0.25$, so the noise variance is 0.09 (`var = 0.09`). What would happen if you set `var = 0.001`?

Exercise 5 — Support Vector Machines

(a) The maximal margin classifier solves

$$\min_{\beta_0,\boldsymbol{\beta}} \frac{1}{2}\|\boldsymbol{\beta}\|^2 \quad \text{subject to} \quad y_i(\mathbf{x}_i^\top \boldsymbol{\beta} + \beta_0) \geq 1, \quad i = 1, \ldots, n.$$

Explain what the **margin** is and why we want to maximise it. What are **support vectors** and which KKT condition identifies them?

(b) The support vector classifier (SVC) introduces slack variables $\epsilon_i \geq 0$ and a cost parameter C:

$$\max_{\beta_0,\boldsymbol{\beta},\boldsymbol{\epsilon}} M \quad \text{subject to} \quad y_i(\beta_0 + \boldsymbol{\beta}^\top \mathbf{x}_i) \geq M(1-\epsilon_i), \quad \sum_{i=1}^{n} \epsilon_i \leq C.$$

Describe the role of C. What happens to the number of support vectors and the margin width as C increases?

(c) The SVM extends the SVC to non-linear boundaries using the kernel trick. The dual decision function is

$$f(\mathbf{x}) = \beta_0 + \sum_{i=1}^{n} \alpha_i K(\mathbf{x}, \mathbf{x}_i).$$

Explain why we can replace $\langle \phi(\mathbf{x}), \phi(\mathbf{x}_i) \rangle$ with $K(\mathbf{x}, \mathbf{x}_i)$, and give one example of a kernel that implicitly maps to an infinite-dimensional feature space.

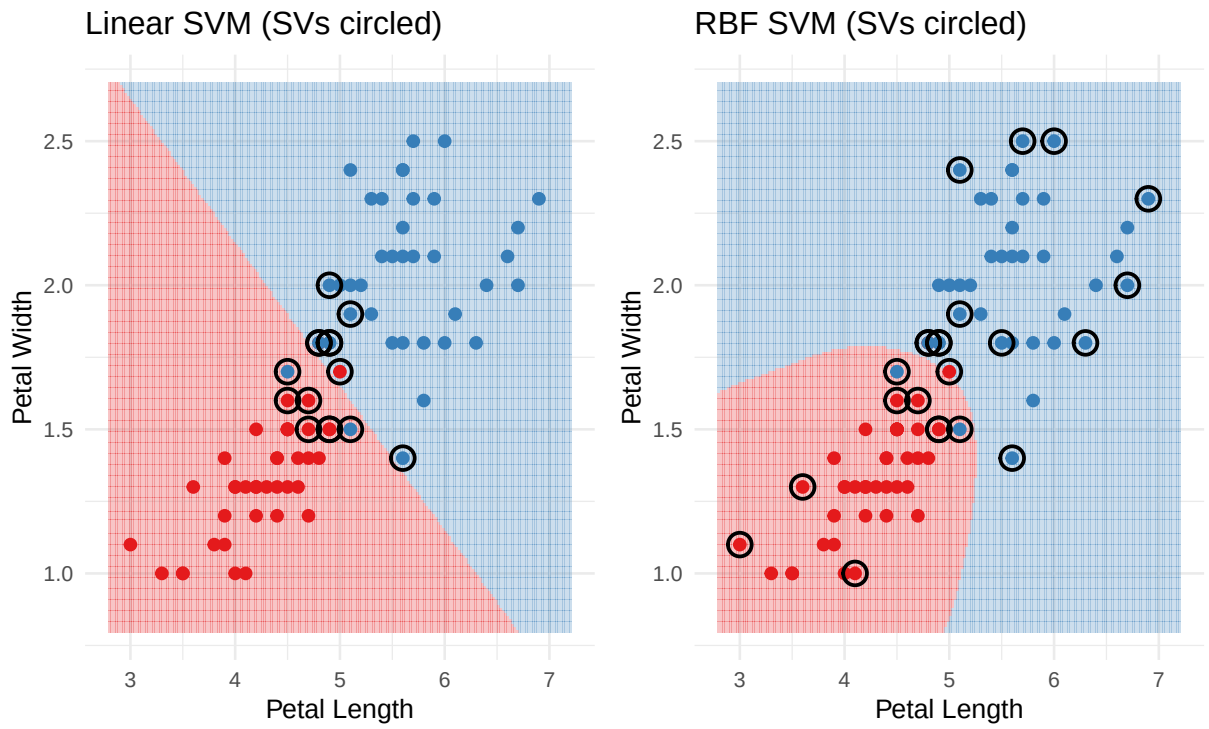
(d) The code below fits linear and RBF kernel SVMs to a two-class subset of the `iris` data and plots the decision boundaries. Run the code and compare the two classifiers in terms of boundary shape, number of support vectors, and test accuracy.

```
library(kernlab)

iris2 <- subset(iris, Species != "setosa")
iris2$Species <- factor(iris2$Species)

set.seed(5)
idx      <- sample(nrow(iris2), 80)
tr       <- iris2[idx, ];   te <- iris2[-idx, ]

svm_lin <- ksvm(Species ~ Petal.Length + Petal.Width,
                data = tr, kernel = "vanilladot", C = 1)
```

```
##  Setting default kernel parameters
```

```
svm_rbf <- ksvm(Species ~ Petal.Length + Petal.Width,
                data = tr, kernel = "rbfdot",
                kpar = list(sigma = 1), C = 1)

# Prediction grid
g <- expand.grid(
  Petal.Length = seq(2.8, 7.2, length.out = 200),
  Petal.Width  = seq(0.8, 2.7, length.out = 200))
g$lin <- predict(svm_lin, g)
g$rbf <- predict(svm_rbf, g)

sv_lin <- tr[alphaindex(svm_lin)[[1]], ]
sv_rbf <- tr[alphaindex(svm_rbf)[[1]], ]

p_lin <- ggplot() +
  geom_tile(data = g, aes(Petal.Length, Petal.Width, fill = lin),
            alpha = 0.25) +
  geom_point(data = tr,
             aes(Petal.Length, Petal.Width, colour = Species), size = 1.8) +
  geom_point(data = sv_lin,
             aes(Petal.Length, Petal.Width),
             shape = 21, size = 3.5, stroke = 1, colour = "black") +
  scale_fill_brewer(palette = "Set1") +
```

```
  scale_colour_brewer(palette = "Set1") +
  labs(title = "Linear SVM (SVs circled)",
       x = "Petal Length", y = "Petal Width") +
  theme_minimal(base_size = 10) + theme(legend.position = "none")

p_rbf <- ggplot() +
  geom_tile(data = g, aes(Petal.Length, Petal.Width, fill = rbf),
            alpha = 0.25) +
  geom_point(data = tr,
             aes(Petal.Length, Petal.Width, colour = Species), size = 1.8) +
  geom_point(data = sv_rbf,
             aes(Petal.Length, Petal.Width),
             shape = 21, size = 3.5, stroke = 1, colour = "black") +
  scale_fill_brewer(palette = "Set1") +
  scale_colour_brewer(palette = "Set1") +
  labs(title = "RBF SVM (SVs circled)",
       x = "Petal Length", y = "Petal Width") +
  theme_minimal(base_size = 10) + theme(legend.position = "none")

library(patchwork)
p_lin + p_rbf
```




```
# Test accuracy
cat("Linear SVM test accuracy:",
    round(mean(predict(svm_lin, te) == te$Species), 3), "\n")
```

```
## Linear SVM test accuracy: 0.95
```

```
cat("RBF SVM test accuracy:   ",
    round(mean(predict(svm_rbf, te) == te$Species), 3), "\n")
```

```
## RBF SVM test accuracy:    0.95
```

```r
cat("Linear SVM support vectors:", nSV(svm_lin), "\n")
```

```
## Linear SVM support vectors: 15
```

```r
cat("RBF SVM support vectors:   ", nSV(svm_rbf), "\n")
```

```
## RBF SVM support vectors:    23
```

(e) SVM is naturally a binary classifier. Describe the two standard strategies (**one-vs-one** and **one-vs-all**) for extending SVM to $K > 2$ classes, and state when each is recommended.